

Collection of Pseudo-Code definitions for the Tensor Omics Implementation

Asis Hallab, “el Bobito”

May 20, 2025

1 F42 Core Sorting Module Pseudocode

1.1 Module Overview

The sorting module provides type-specific, in-place sorting routines for 1D arrays of integers, real numbers, and character sequences. It uses Fortran `interface` blocks to overload a common sort function name across types. All working memory is preallocated and passed explicitly.

1.2 Type Definitions

- No new types are defined.
- Only standard Fortran arrays are used.

1.3 Public Interface

- `interface sort_array`
 - `module procedure sort_integer, sort_real, sort_character`

Each sort routine works in-place and expects preallocated working memory.

1.4 Function Signatures

- `subroutine sort_integer(array, work)`
 - `integer, dimension(n), intent(inout) :: array`
 - `integer, dimension(n), intent(inout) :: work` (temporary work array)
- `subroutine sort_real(array, work)`
 - `real, dimension(n), intent(inout) :: array`
 - `integer, dimension(n), intent(inout) :: work`
- `subroutine sort_character(array, work)`
 - `character(len=*), dimension(n), intent(inout) :: array`
 - `integer, dimension(n), intent(inout) :: work`

1.5 Algorithm: General Structure (All Types)

1. Initialize permutation array:

$$\text{work}(i) = i \quad \text{for } i = 1, \dots, n$$

2. Sort the permutation array **work** based on the corresponding values in **array**.

- Use insertion sort (small n) or quicksort (large n).

3. Rearrange **array** according to the sorted permutation:

- Use a cycle leader algorithm to avoid extra memory allocation.

1.6 Details for Each Type

- **Integer**: Compare **array**(i) and **array**(j) numerically.
- **Real**: Compare **array**(i) and **array**(j) numerically.
- **Character**: Compare **array**(i) and **array**(j) lexicographically.

1.7 Memory Requirements

- For sorting an array of size n :
 - One integer work array of size n is needed.

1.8 Algorithm: Non-Recursive Quicksort for Permutation Array

To comply with F42 and WebAssembly restrictions, Quicksort is implemented without recursion, using an explicit manual stack structure for subarray boundaries.

1.8.1 Procedure: quicksort_nonrecursive

- subroutine quicksort_nonrecursive(array, perm, n, stack_left, stack_right)
 - type depends on context :: array (integer, real, or character array)
 - integer, dimension(n), intent(inout) :: perm (permutation array)
 - integer, intent(in) :: n (size of array)
 - integer, dimension(:), intent(inout) :: stack_left, stack_right (manual stack arrays)

1.8.2 Working Memory Requirements

- **stack_left** and **stack_right**: integer arrays of length at least $\log_2(n) + 10$ (safe overestimate).

1.8.3 Steps

1. Initialize:

- Set **stack_ptr** = 1.
- Push initial full range: **stack_left**(1) = 1, **stack_right**(1) = n.

2. Main loop:

- While **stack_ptr** > 0:
 - (a) Pop:
 - Set **left** = **stack_left**(**stack_ptr**), **right** = **stack_right**(**stack_ptr**).

- Decrement `stack_ptr`.
- (b) If `left ≥ right`, continue to next loop iteration.
- (c) Select pivot:
 - Set `pivot_index = (left + right) / 2`.
 - Set `pivot_value = array(perm(pivot_index))`.
- (d) Partition:
 - Initialize `i = left, j = right`.
 - While `i ≤ j`:
 - * Increment `i` while `array(perm(i)) < pivot_value`.
 - * Decrement `j` while `array(perm(j)) > pivot_value`.
 - * If `i ≤ j`:
 - Swap `perm(i)` and `perm(j)`.
 - Increment `i`, decrement `j`.
- (e) Push new ranges onto the stack:
 - If `left < j`:
 - * Increment `stack_ptr`.
 - * Set `stack_left(stack_ptr) = left, stack_right(stack_ptr) = j`.
 - If `i < right`:
 - * Increment `stack_ptr`.
 - * Set `stack_left(stack_ptr) = i, stack_right(stack_ptr) = right`.

1.8.4 Notes

- No recursion: stack is manually managed.
- Memory predictable: all working memory passed as arguments.
- Stateless: no hidden or module-wide variables.
- Safe for WebAssembly compilation and fully F42 compliant.

2 Unit Tests for F42 Core Sorting Module

The following unit tests verify correctness, robustness, and F42 compliance of the non-recursive sorting routines for integer, real, and character arrays. Each test assumes that all required working memory is preallocated before calling sort procedures.

2.1 Test 1: Sort Ascending Integer Array

- Input:
 - `array = [5, 2, 9, 1, 6]`
 - `work` = preallocated integer array of size 5
- Action:
 - Call `sort_array(array, work)`.
- Expected:
 - `array = [1, 2, 5, 6, 9]`

2.2 Test 2: Sort Descending Real Array

- Input:
 - `array = [3.5, 2.2, 8.8, 1.1, 7.7]`
 - `work =` preallocated integer array of size 5
- Action:
 - Call `sort_array(array, work)`.
- Expected:
 - `array = [1.1, 2.2, 3.5, 7.7, 8.8]`

2.3 Test 3: Sort Random Character Array

- Input:
 - `array = ["dog", "apple", "zebra", "cat", "bird"]`
 - `work =` preallocated integer array of size 5
- Action:
 - Call `sort_array(array, work)`.
- Expected:
 - `array = ["apple", "bird", "cat", "dog", "zebra"]`

2.4 Test 4: Stability on Already Sorted Arrays

- Input:
 - `array = [1, 2, 3, 4, 5]`
 - `work =` preallocated integer array of size 5
- Action:
 - Call `sort_array(array, work)`.
- Expected:
 - `array` remains `[1, 2, 3, 4, 5]`

2.5 Test 5: Behavior on Empty Array

- Input:
 - `array = []`
 - `work =` preallocated integer array of size 0
- Action:
 - Call `sort_array(array, work)`.
- Expected:
 - No error or crash; operation completes trivially.

2.6 Test 6: Correctness of Sorting After Random Shuffles

- Input:
 - Generate random permutation of integers [1,2,3,...,1000].
 - `array` = shuffled integers
 - `work` = preallocated integer array of size 1000
- Action:
 - Call `sort_array(array, work)`.
- Expected:
 - `array` = [1, 2, 3, ..., 1000]

3 F42 Adaptive Gaussian k-Nearest Neighbor Smoothing (“Marduk”)

In Assyrian mythology, *Marduk* is known as the “placer of stars” due to his role in creating the heavens and earth, including the placement of constellations and the stars themselves.

3.1 Vector-Valued Smoothing Routine `adapt_gauss_knn_smooth`

This routine performs local smoothing of a vector-valued field (e.g., expression or shift vectors) by computing a Gaussian-weighted average over the k nearest neighbors of a given target point. The kernel width σ is adapted for each target point based on the distance to its k -th nearest neighbor. The smoothed vector is written to the output `result`.

Function Signature

```
subroutine adapt_gauss_knn_smooth(  
    vecs,           ! real(ndim, n_points)  
    inds,           ! logical(n_points)  
    target_index,   ! integer  
    k,              ! integer  
    kd_tree,        ! prebuilt K-D tree structure  
    scratch_indices, ! integer(k)  
    scratch_dists,   ! real(k)  
    result          ! real(ndim)  
)
```

Argument Explanation

- `vecs`: A `real(ndim, n_points)` matrix containing the vector field to be smoothed. Each column is a vector in $\mathbb{R}^{ndim}$.
- `inds`: A logical mask array of length `n_points`. Only entries with `.true.` are considered valid data points for smoothing.
- `target_index`: The index in `vecs` of the point to smooth. This must be one of the `.true.` entries in `inds`.
- `k`: The number of nearest neighbors to use.
- `kd_tree`: A precomputed K-D tree for `vecs(:, inds)`, assumed to support masked neighbor lookup.

- `scratch_indices`: Transient integer array of length at least `k` for neighbor indices.
- `scratch_dists`: Transient real array of length `k` for distances to the neighbors.
- `result`: Output array of dimension `ndim` containing the smoothed vector.

Note: This function produces the smoothed vector only for the single `target_index`. To smooth an entire subfield, the function must be called iteratively for all entries with `.true.` in `inds`.

Pseudocode

```

1. target_coords := vecs(:, target_index)

2. call find_k_nearest_neighbors_masked(
    kd_tree,
    target_coords,
    inds,
    k,
    scratch_indices,
    scratch_dists
)

3. d_max := scratch_dists(k)
   sigma := d_max / 3.0

4. result := 0.0_vector(ndim)
   total_weight := 0.0

5. do i = 1, k
    dist := scratch_dists(i)
    w := exp( - (dist**2) / (2.0 * sigma**2) )

    idx := scratch_indices(i)
    result := result + w * vecs(:, idx)
    total_weight := total_weight + w
end do

6. if total_weight > 0.0 then
    result := result / total_weight
else
    result := vecs(:, target_index) ! fallback: no smoothing
end if

```

Usage Pattern for Full Subfield Smoothing

To smooth all vectors in a subfield defined by `inds`, iterate over all `target_index` such that `inds(target_index) = .true.`, and call `adapt_gauss_knn_smooth` for each.

This guarantees full F42 compatibility, vector-wise smoothing, and modular use with Tensor Omics vector field data structures.

3.2 Applications Across TOX

- Normalization: Fit for gene-wise SD vs. mean.
- Interpolation and smoothing of expression vectors or shift vectors.
- Lexical smoothing across metadata axes.

- Clock hand and trajectory smoothing (e.g. Mjölnir-aligned).
- Local field smoothing in GAWD or Bobito cones.

3.3 Test Suite for `adapt_gauss_knn_smooth`

This suite verifies the correctness of the adaptive Gaussian k-nearest neighbor smoothing algorithm across edge cases, synthetic inputs, and known outcomes.

Test 1: Single Point, Self Only

Goal: Verify that smoothing a vector when only one point is available returns that point unchanged.

Input:

```
vecs := [1.0, 2.0, 3.0] ! ndim = 3, n_points = 1
inds := [.true.]
target_index := 1
k := 1
result := [0.0, 0.0, 0.0]
```

Expected Output:

```
result == [1.0, 2.0, 3.0]
```

Test 2: Two Identical Vectors, $k = 2$

Goal: Check that smoothing over two identical vectors returns the same vector.

Input:

```
vecs := [[1.0, 1.0],
          [2.0, 2.0],
          [3.0, 3.0]] ! ndim = 3, n_points = 2
inds := [.true., .true.]
target_index := 1
k := 2
```

Expected Output:

```
result == [1.0, 2.0, 3.0] ! unchanged because all neighbors identical
```

Test 3: Linear Interpolation Between Two Vectors

Goal: Validate that Marduk performs linear interpolation for symmetric neighbors.

Input:

```
vecs := [[0.0, 2.0],
          [0.0, 2.0],
          [0.0, 2.0]] ! ndim = 3, n_points = 2
inds := [.true., .true.]
target_index := 1
k := 2
```

Expected Output:

```
result approximately equals [1.0, 1.0, 1.0] ! mean of two equidistant neighbors
```

Test 4: Gaussian Weight Decay Check

Goal: Ensure closer neighbor contributes more via Gaussian kernel.

Input:

```
vecs := [[0.0, 10.0],
          [0.0, 10.0],
          [0.0, 10.0]]
inds := [.true., .true.]
target_index := 1
k := 2
```

Expected Output:

```
result closer to [0.0, 0.0, 0.0] than to [10.0, 10.0, 10.0]
```

Test 5: Masked Points Exclusion

Goal: Confirm that `.false.` entries in `inds` are never used.

Input:

```
vecs := [[0.0, 10.0, 20.0],
          [0.0, 10.0, 20.0],
          [0.0, 10.0, 20.0]]
inds := [.true., .false., .true.]
target_index := 1
k := 2
```

Expected Output:

```
result is mean of point 1 and point 3, not including masked-out point 2
```

Test 6: Zero Neighbors Edge Case

Goal: Verify fallback behavior if no valid neighbors found.

Input:

```
vecs := [[1.0], [2.0], [3.0]]
inds := [.false.] ! target not in mask
target_index := 1
k := 1
```

Expected Output:

```
result == vecs(:, 1) ! fallback to unsmoothed value
```

Test 7: Dimensional Scaling Check

Goal: Confirm algorithm generalizes to arbitrary `ndim`.

Input:

```
vecs := real(10, 100) ! ndim = 10, n_points = 100
Populate with smooth gradient vectors
inds := all .true.
```



```
target_index := random index
k := 10
```

Expected Output:

result approximately equals smooth interpolation within local 10D neighborhood

3.4 The which Function for Logical Arrays (F42-Compliant)

The `which` function extracts the indices of all entries in a logical array that evaluate to `.TRUE.`. This routine is the direct analogue of the R function `which()`, designed to work with logical arrays resulting from any Fortran logical expression (e.g., `x > 3.0 .AND. MOD(x,2) == 0`). It follows the F42 principles: stateless, allocation-free, and usable in high-performance settings.

3.4.1 Interface Specification

```
subroutine which(
    mask(n),           ! Input logical array
    n,                 ! Size of the input array
    idx_out(m_max),    ! Preallocated output array of indices
    m_max,             ! Capacity of idx_out
    m_out              ! Number of matches returned
)
```

- `mask(n)`: Logical array derived from a Fortran logical expression
- `idx_out(m_max)`: Output array holding 1-based indices of matches
- `m_out`: Actual number of matches written to `idx_out`

3.4.2 Pseudocode Implementation

```
m_out = 0
do i = 1, n
    if (mask(i)) then
        if (m_out < m_max) then
            m_out = m_out + 1
            idx_out(m_out) = i
        endif
    endif
enddo
```

This implementation ensures no allocation and full compatibility with high-performance computing contexts (e.g., OpenMP, SIMD, or GPU-friendly flattening).

3.4.3 Test Case 1: Simple Threshold Filter

- **Input:** `x = [1.0, 4.0, 6.0, 2.0, 8.0]`
- **Logical Mask:** `mask = (x > 3.0)`
- **Expected Output:** `idx_out = [2, 3, 5], m_out = 3`

3.4.4 Test Case 2: Composite Logical Condition

- **Input:** `x = [1.0, 4.0, 6.0, 2.0, 8.0]`
- **Mask:** `mask = (x > 3.0 .AND. MOD(INT(x),2) == 0)`
- **Expected Output:** `idx_out = [2, 5], m_out = 2`

3.4.5 Test Case 3: No Matches

- **Input:** `x = [1.0, 2.0, 3.0]`
- **Mask:** `mask = (x > 10.0)`
- **Expected Output:** `idx_out = [], m_out = 0`

3.4.6 Performance and Compliance Notes

- This function is safe for use in any high-performance loop or parallel region.
- Overflow behavior (when `m_out` exceeds `m_max`) must be handled externally.
- All array bounds and allocations are managed by the caller.

4 F42-Compliant Index-Based K-D Trees and BSTs

4.1 Overview

This section defines efficient, allocation-free, recursion-free, and Fortran-compatible implementations of:

- A 1D Binary Search Tree (BST) represented as a flat index array
- A Cartesian K-D Tree using descending variance dimension order
- A Spherical K-D Tree (used for directional queries based on cosine similarity)
- Core access and query routines
- Unit tests for each structure

All memory structures are passed in. The trees are represented purely as sorted index arrays. No structs, recursion, or dynamic memory is used.

4.2 Binary Search Tree (BST) Construction via Sorting

Input:

- Real array `x(n)` — the data to be indexed
- Integer array `ix(n)` — index array to be filled

Pseudocode:

```
subroutine build_bst_index(x, n, ix)
  real, intent(in) :: x(n)
  integer, intent(out) :: ix(n)

  call quicksort_indices(x, ix, n)
end subroutine
```

4.3 Access Functions for BST (Flat Index Array)

Get Value at Sorted Position Returns the value of the original data array at the position given by the sorted index.

```
function get_sorted_value(x, ix, i) result(val)
real, intent(in) :: x(:)
integer, intent(in) :: ix(:)
integer, intent(in) :: i
real :: val

val = x(ix(i))
end function
```

Get Sorted Index for Range Query Example: Find values in x between a given lo and hi . This function returns a list of positions i such that $lo \leq x(ix(i)) \leq hi$.

```
subroutine bst_range_query(x, ix, n, lo, hi, out_ix, out_n)
real, intent(in) :: x(:)
integer, intent(in) :: ix(:)
integer, intent(in) :: n
real, intent(in) :: lo, hi
integer, intent(out) :: out_ix(n)
integer, intent(out) :: out_n

integer :: i
out_n = 0

do i = 1, n
if (x(ix(i)) >= lo .and. x(ix(i)) <= hi) then
out_n = out_n + 1
out_ix(out_n) = ix(i)
else if (x(ix(i)) > hi) then
exit ! Since array is sorted, we can stop early
end if
end do
end subroutine
```

Use Case Example Sort an array and extract all values between 2.0 and 3.5.

```
real :: x(1000)
integer :: ix(1000), res_ix(1000), res_n

call random_fill(x)
call build_bst_index(x, 1000, ix)
call bst_range_query(x, ix, 1000, 2.0, 3.5, res_ix, res_n)
! Now x(res_ix(1:res_n)) holds the sorted matching values
```

4.4 Cartesian K-D Tree Construction

Input:

- Real matrix $X(d, n)$ — the data points
- Integer array $kd_ix(n)$ — output index array representing K-D Tree

- Integer array `dim_order(d)` — precomputed dimension order by descending variance
- Integer work arrays for temporary use

Pseudocode:

```

subroutine build_kd_index(X, d, n, kd_ix, dim_order, work)
  ! Loop-based K-D Tree construction by sorting and partitioning
  ! No recursion. Uses stack to manage index intervals and split dimension.

  integer :: stack_top, stack(1:2, max_depth)
  integer :: l, r, dim, mid, depth, i

  stack_top = 1
  stack(:, 1) = [1, n]

  do while (stack_top > 0)
    l = stack(1, stack_top)
    r = stack(2, stack_top)
    stack_top = stack_top - 1
    if (r <= l) cycle

    depth = some_depth_heuristic(l, r) ! Or simply count levels
    dim = dim_order(mod(depth, d) + 1)

    call partial_sort_by_dimension(X, d, kd_ix, l, r, dim)
  end do
end subroutine

```

4.5 Spherical K-D Tree for Directional Queries

Input:

- Normalized vector matrix $V(d, n)$ — each column is unit length
- Integer array `sphere_ix(n)` — output index array for spherical K-D Tree

Pseudocode:

```

subroutine build_spherical_kd(V, d, n, sphere_ix, dim_order)
  call build_kd_index(V, d, n, sphere_ix, dim_order)
end subroutine

```

4.6 Access Routines

Get value at sorted position:

```

function get_value_sorted(x, ix, i)
  real :: get_value_sorted
  real, intent(in) :: x(:)
  integer, intent(in) :: ix(:)
  get_value_sorted = x(ix(i))
end function

```

Get point from KD index:

```
subroutine get_kd_point(X, kd_ix, i, point)
  real, intent(in) :: X(:, :)
  integer, intent(in) :: kd_ix(:)
  real, intent(out) :: point(:)
  point = X(:, kd_ix(i))
end subroutine
```

4.7 Unit Tests

BST Test:

```
call random_array(x, n)
call build_bst_index(x, n, ix)
assert_monotonic(x(ix))
```

K-D Tree Test:

```
call random_matrix(X, d, n)
call compute_dim_order(X, d, n, dim_order)
call build_kd_index(X, d, n, kd_ix, dim_order)
assert_valid_kd_structure(kd_ix)
```